

Mục lục

Phần 1. Thông tin chung	1
Phần 2. Sơ lược về dự án web sàn tuyển dụng tích hợp mini-ATS	3
2.1 Giới thiệu về miCareer	3
2.2 Tính phù hợp của hệ thống với kiến trúc RAG	3
Phần 3. Nâng cấp CSDL phục vụ tích hợp AI	4
Phần 4. Kiến trúc dự kiến của hệ thống.....	6
4.1 Các phương án	6
4.2 kiến trúc tổng thể.....	6
4.2.1 Xử lý dữ liệu đầu vào	7
4.2.2 Tìm kiếm/Truy xuất tri thức	8
4.2.3 Tổng hợp dữ liệu và sinh câu trả lời	8

Phần 1. Thông tin chung

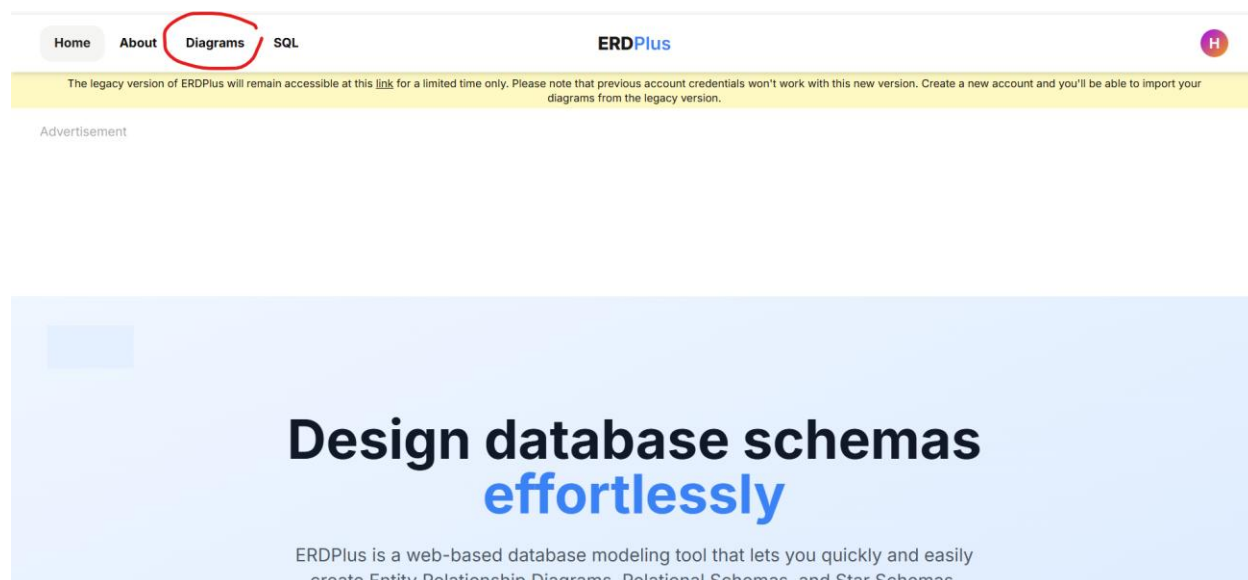
Tên đề tài dự kiến: Thiết kế và triển khai AI Layer dựa trên kiến trúc Retrieval-Augmented Generation (RAG) tích hợp vào hệ thống quản lý tuyển dụng

Đối với đề tài này, mình dự kiến sẽ triển khai trên một CSDL đã được mình thiết kế sẵn để dùng cho môn lập trình web và nhập môn AI.

Khi gửi tài liệu này mình sẽ gửi kèm:

- 1 file .erdplus tên ITJobMarket_MiniATS_ERD là bản vẽ gốc của web tuyển dụng
- 1 file .erdplus tên ITJobMarket_MiniATS_ERD_TTCS là bản vẽ ERD đã nâng cấp của dự án Web để tích hợp hệ thống AI
- 1 file .docx là báo cáo môn CSDL PT của mình, tài liệu này bao gồm phân tích nghiệp vụ + thiết kế + chuẩn hóa CSDL (bỏ qua triển khai vật lý) cho dự án Web (chưa nâng cấp)
- 1 video youtube của kênh sydexa về hệ thống AI dựa trên kiến trúc RAG của Uber. Đây chính là nguồn gốc của ý tưởng và là nguồn cảm hứng của mình về đề tài này. Link tới video: <https://youtu.be/O7yxSi8quA0?si=V1h90ByP3AjSFMt1>

các bạn hãy tải về và vào trang web <https://erdplus.com/> bấm vào mục Diagrams và chọn import file vừa tải.



HomeAboutDiagramsSQL

ERDPlus

H

New folderRenameDelete

Search diagrams...

New diagramImportOrganize

Diagrams

Trash

Diagrams

ER DiagramRelational Schema

11 Sort

ITJobMarket_MiniATS_ERD_TTCS

ER Diagram

17 hours ago

...

ITJobMarket_MiniATS_ERD

ER Diagram

4 days ago

...

ITJobMarket_MiniATS_ERD-Relational Schema

Relational Schema

5 days ago

...

ITJobMarket_MiniATS_ERD_trimmed

ER Diagram

6 days ago

...

Advertisement

Phần 2. Sơ lược về dự án web sàn tuyển dụng tích hợp mini-ATS

2.1 Giới thiệu về miCareer

Dự án miCareer là hệ thống được kết hợp giữa sàn tuyển dụng IT và hệ thống quản trị tuyển dụng thu nhỏ. Hệ thống được thiết kế để hỗ trợ quy trình tuyển dụng từ lúc tin tuyển dụng được đăng cho đến lúc ứng viên được ký hợp đồng lao động. Cụ thể qua các bước như: Công ty đăng tin tuyển dụng - ứng viên nộp đơn – HR chọn và lên lịch phỏng vấn – đàm phán Offer.

2.2 Tính phù hợp của hệ thống với kiến trúc RAG

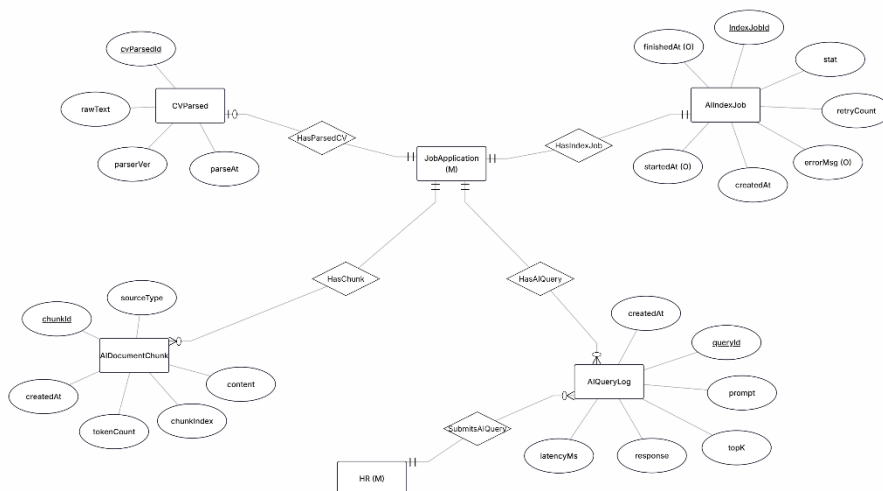
miCareer được thiết kế lưu trữ một lượng lớn các dữ liệu phi cấu trúc mang giá trị thông tin cao, tạo điều kiện lý tưởng cho RAG phát huy tác dụng. Các nguồn rich-text điển hình có thể khai thác như:

- JobPosting.desc: Mô tả dạng text/HTML về một tin tuyển dụng
- Candidate.bio: Thông tin tiểu sử về ứng viên
- JobApplication.cvSnapUrl: Đường dẫn tới CV snapshot của ứng viên tại thời điểm ứng tuyển. Đây là thông tin quan trọng nhất và sẽ được parse ra text sau đó được xử lý để làm dữ liệu cho LLM
- InterviewFeedBack.cmt: Nhận xét chi tiết từ một người phỏng vấn về ứng viên cho một buổi phỏng vấn.
- Ngoài ra có thể dùng đến Offer.desc (các quyền lợi/ điều khoản của Offer) và EmailLog.content (Snapshot nội dung Email đã gửi đi)

Trên đó chỉ là các nguồn điển hình, khi triển khai thực tế mình dự tính sẽ sử dụng cả các trường liên quan của một thực thể ví dụ như : JobPosting.desc sẽ nên đi kèm với title, minSalary, maxSalary, workMode v.v

Phần 3. Nâng cấp CSDL phục vụ tích hợp AI

Mình dự kiến sẽ thêm vào 4 bảng, biểu diễn trên ERDPlus như sau:



- Chú thích: JobApplication và HR được đánh dấu “(M)” là mượn từ ERD nghiệp vụ chính của WEB.

Mô tả cho phần này (Khúc này lười quá GPT nhé):

Nhằm tích hợp khả năng hỗ trợ phân tích hồ sơ ứng tuyển dựa trên kiến trúc Retrieval-Augmented Generation (RAG), hệ thống được mở rộng thêm một AI Layer độc lập với các thực thể nghiệp vụ cốt lõi.

Nguyên tắc thiết kế được tuân thủ gồm:

- Không sửa đổi cấu trúc core ATS hiện tại
- Không phá vỡ mức chuẩn hóa 3NF
- Không trộn logic AI vào các bảng nghiệp vụ
- Tách biệt hoàn toàn AI Layer khỏi business core

AI Layer được thiết kế xoay quanh thực thể trung tâm JobApplication, đảm bảo mỗi hồ sơ ứng tuyển là một bounded context độc lập phục vụ quá trình ingestion và truy vấn tri thức.

Bốn thực thể mới được bổ sung gồm:

- AllIndexJob: Quản lý queue và trạng thái ingestion (Queued, Processing, Success, Failed).
- CVParsed: Lưu trữ nội dung CV snapshot sau khi parse tại ingestion-time.
- AIDocumentChunk: Lưu các đoạn văn bản đã được chunk phục vụ retrieval trong RAG.
- AIQueryLog: Ghi nhận toàn bộ truy vấn AI do HR thực hiện nhằm phục vụ audit và phân tích học thuật.

Mối quan hệ giữa các thực thể được thiết kế theo nguyên tắc non-identifying, mỗi bảng đều có khóa chính riêng và liên kết tới JobApplication thông qua khóa ngoại.

Hệ thống đảm bảo:

- Mỗi JobApplication có đúng một AllIndexJob
- Mỗi JobApplication có tối đa một CVParsed
- Một JobApplication có thể sinh nhiều AIDocumentChunk
- Một HR có thể thực hiện nhiều AIQueryLog

Thiết kế này giúp AI Layer hoạt động độc lập, có thể mở rộng hoặc loại bỏ mà không ảnh hưởng đến cấu trúc nghiệp vụ chính của hệ thống ATS.

Phần 4. Kiến trúc dự kiến của hệ thống

4.1 Các phương án

Tính đến hiện tại mình có 2 phương án cho đề tài này:

1. Hệ thống sẽ xoay quanh một JobApplication để HR có thể prompt với LLM. Tức là HR sẽ xem một đơn ứng tuyển cụ thể và có thể hỏi như:
 - a. Tóm tắt CV ứng viên
 - b. So sánh mức độ phù hợp giữa CV và JobDescription
 - c. Tổng hợp nhận xét sau phỏng vấn về ứng viên
 - d. Và tất cả các câu hỏi khác dựa trên bộ dữ liệu CV ứng viên + JD của JobPosting + các Feedback sau phỏng vấn + nội dung các Email đã gửi + nội dung các phiếu Offer.
2. Hệ thống sẽ xoay quanh một JobPosting để HR có thể hỏi các câu hỏi ở mức độ một đơn tuyển dụng cụ thể kiểu như:
 - a. Có bao nhiêu đơn ứng tuyển vào JobPosting này, liệt kê sơ bộ tất cả các ứng viên
 - b. So sánh giữa các ứng viên và sắp xếp theo các tiêu chí đề xuất
 - c. Thống kê tất cả các thông số kiểu như tỷ lệ được vào vòng phỏng vấn, tỷ lệ Offer, mức chênh của Offer ứng viên – Offer từ HR, số ứng viên được ký hợp đồng / tổng số đơn ứng tuyển v.v

Trong bản tổng quan này mình sẽ tập trung vào phương án thứ nhất, một hệ thống như vậy đã đủ phức tạp để làm trong 6 tuần. Nếu sau này có dư thời gian thì nâng cấp DB lần nữa để thêm phương án 2 cũng chưa muộn

Vì vậy các giai đoạn liệt kê dưới đây là dựa trên phương án 1.

4.2 kiến trúc tổng thể

Gồm 3 giai đoạn chính:

- Xử lý dữ liệu đầu vào
- Tìm kiếm/truy xuất tri thức
- Tổng hợp dữ liệu và sinh câu trả lời

4.2.1 Xử lý dữ liệu đầu vào

Ở góc độ ứng viên thì mình dự tính sẽ làm một UI đơn giản và thực hiện duy nhất việc sau:

- Đăng nhập (với các tài khoản, thông tin mặc định insert sẵn, không làm UI phần này)
- Xem danh sách JobPosting (tất cả các JobPosting cũng được tạo bằng insert DB)
- Xem một JobPosting, nộp CV + coverletter

Sau khi ứng viên đã nộp đơn, hệ thống tạo một JobApplication. Sau đó một job sẽ được đưa vào queue để xử lý theo luồng như sau:

1. Parse CV SnapShot thành json (kèm raw text)
2. Hệ thống tổng hợp các dữ liệu tĩnh (có sẵn tại thời điểm apply) gồm:
 - a. CV parsed
 - b. Cover letter (nếu có)
3. Chia khối dữ liệu thành các chunk
4. Embed từng chunk
5. Lưu vào vector storage

Ở đây các dữ liệu tĩnh và dài mới được embed, các dữ liệu có sau này như thông tin về phỏng vấn, Offer, email sẽ được kéo trực tiếp khi HR prompt.

Cập nhật 9:46 PM 4/3/2026:

- CV snapshot sẽ được parse thành JSON theo chuẩn pydantic định nghĩa tại `models/cv_models.py` trong dự án FANG + raw text
- JobPosting.desc là thông tin chung cho nhiều đơn ứng tuyển -> không parse mà gán trực tiếp vào context lúc HR prompt
- Tại sao không thể sử dụng luôn JSON để chunk mà phải chuyển thành markdown?
 - Bản chất huấn luyện của mô hình Embedding: Các mô hình nhúng (như OpenAI text-embedding-3, Google text-embedding-gecko hay các model open-source) được huấn luyện chủ yếu trên ngôn ngữ tự nhiên (sách báo, Wikipedia, các bài viết trên internet). Chúng hiểu cực tốt sự liên kết ngữ nghĩa giữa một tiêu đề (Heading) và đoạn văn bản bên dưới nó. Trong khi đó, JSON là cấu trúc dữ liệu cho máy đọc, mô hình sẽ bị "phân tâm" bởi các ngoặc nhọn {}, ngoặc vuông [], và các từ khóa lặp lại.
 - Tiết kiệm Token (Chi phí & Hiệu năng): JSON chứa rất nhiều ký tự cấu trúc (noise). Ví dụ thay vì tốn token cho chuỗi `{"experience": [{"company": "ABC", "role": "Dev"}]}`, Markdown chỉ cần `### Kinh nghiệm\n- **Dev**`

tại ABC'. Giảm token rác giúp Vector DB hoạt động nhẹ nhàng hơn và tăng mật độ thông tin có ích trong mỗi chunk.

- An toàn khi "Cắt" (Chunking): Khi cắt một đoạn JSON quá dài, rất dễ cắt ngang giữa chừng một object (ví dụ cắt mất dấu đóng ngoặc }). Nếu đưa một đoạn JSON bị cắt dở vào Prompt Context ở pha Retrieval, LLM đôi khi sẽ bị bối rối. Cắt Markdown theo các thẻ Heading (MarkdownHeaderTextSplitter) đảm bảo các chunk luôn giữ được trọn vẹn ngữ nghĩa và dễ đọc.

4.2.2 Tìm kiếm/Truy xuất tri thức

HR cũng có một UI rất đơn giản chỉ để xem các JobPosting mà HR này phụ trách. HR sẽ chọn một JobApplication trong một JobPosting và thực hiện hỏi đáp trong này.

Khi HR gửi prompt, hệ thống sẽ trải qua các bước sau:

1. Kiểm tra trạng thái xử lý của dữ liệu, nếu chưa xử lý xong thì không được prompt
2. Khi prompt của HR được chấp nhận gửi đi, hệ thống sẽ embed prompt này
3. Dựa vào vector đó hệ thống sẽ truy xuất thông tin tĩnh dự kiến dùng các thuật toán:
 - a. Cosine similarity
 - b. Top-k chunk
4. Ngoài truy xuất các thông tin tĩnh, hệ thống sẽ truy xuất ngay tại thời điểm đó các thông tin động như:
 - a. InterviewFeedback.cmt
 - b. Offer.desc
 - c. EmailLog.content (nếu cần)

4.2.3 Tổng hợp dữ liệu và sinh câu trả lời

Sau khi đã có được các dữ liệu cần thiết, prompt cuối cùng được xây dựng sẽ gồm:

- Chỉ dẫn hệ thống (System prompt)
- Những chunk (thông tin tĩnh) đã truy xuất được
- Những thông tin truy xuất động
- Prompt từ HR

Trong đó chỉ dẫn hệ thống sẽ yêu cầu LLM trả lời dựa trên context và không làm bất kỳ nhiệm vụ ngoài nghiệp vụ hệ thống nào khác.

Sau cùng prompt tổng sẽ được gửi đến LLM và hệ thống sẽ nhận về câu trả lời (có thể kiểm tra), đẩy ra cho HR và lưu log prompt.